# 중간보고서

# Grad-Deploy v2

Policy-as-Code 기반 K8s 운영 안전망 플랫폼

| | |
|---|---|
| **과목명** | 캡스톤 디자인 |
| **팀명** | 1조 |
| **프로젝트명** | Grad-Deploy |
| **문서 버전** | v2.0 (중간보고서) |
| **작성일** | 2026. 04. 22 |

# 1. 프로젝트 개요 (Background & Why)

## 1.1 배경 및 필요성

**현대 클라우드 인프라의 표준인 Kubernetes(K8s)는 강력한 오케스트레이션 기능을 제공하지만, DevOps 경험이 없는 초보 개발자에게는 '설정 지옥(Configuration Hell)'을 선사합니다. 수많은 YAML 필드 중 단 하나의 오설정이 전체 시스템의 연쇄 장애(Cascading Failure)를 초래하며, 이는 학습 의욕 저하와 운영 비용 폭증으로 이어집니다.**

**실무자 커뮤니티(Reddit r/kubernetes, Hacker News, Kubernetes GitHub Issues)의 장애 사례를 정량 분석한 결과, 문제의 근본 원인은 K8s의 기능 부족이 아닌 '너무 많은 선택지와 설정 오류'로 수렴됩니다. 이에 본 프로젝트는 K8s를 쉽게 만드는 것이 아니라, K8s의 복잡성을 '제한'하는 운영 안전망(Guardrail) 플랫폼을 목표로 합니다.**

## 1.2 문제 정의: 9대 실무 장애 사례 (Data-Driven)

**Reddit, Hacker News, Kubernetes 공식 GitHub Issues를 분석하여 개발자가 반복적으로 경험하는 9대 핵심 장애 패턴을 정의하였습니다. 각 사례는 Grad-Deploy의 가드레일 엔진이 대응하는 근거 데이터입니다.**

| 케이스 ID | 장애명 | 장애 원인 | 영향 범위 | 대응 엔진 |
|---|---|---|---|---|
| Case 1 | Resource Throttling | CPU Limit만 설정, Request 미설정으로 CFS Quota 초과 → 성능 저하 | 서비스 latency 급증, p99 지연 심화 | RA |
| Case 2 | PDB Deadlock | PDB minAvailable ≥ replicas 설정 → 노드 드레인 불가, 업데이트 영구 차단 | 클러스터 유지보수 불능 | GE |
| Case 3 | Probe Loop | Liveness Probe 경로에 DB 연결 체크 포함 → 외부 의존성 장애 시 무한 재시작 | 연쇄 재시작, 서비스 전체 중단 | PG |
| Case 4 | Network Blindness | NetworkPolicy 미설정 → DB 등 내부 자원이 클러스터 외부 또는 불필요한 Pod에 노출 | 보안 침해, 데이터 유출 위험 | NA |
| Case 5 | Config Drift | kubectl edit 등 수동 조작 → Git 저장소와 실 클러스터 상태 불일치, 추적 불능 | 배포 이력 손실, 롤백 불가 | GitOps/CD |
| Case 6 | Finalizer Stuck | 리소스 삭제 시 Finalizer 미해제 → Terminating 상태 5분 이상 고착 | 리소스 영구 잔존, 네임스페이스 삭제 불가 | GE |
| Case 7 | PVC Unbound | StorageClass 미지정 → PVC Pending, Pod 미기동 | DB 서비스 기동 불가 | VE |
| Case 8 | Image Auth Fail | Private Registry imagePullSecrets 누락 → ErrImagePull | 배포 완전 중단 | VE |

| 케이스 ID | 장애명 | 장애 원인 | 영향 범위 | 대응 엔진 |
|---|---|---|---|---|
| Case 9 | Cost Explosion | VM 스펙을 컨테이너에 그대로 적용 → 과도한 Resource Request → 크레딧 조기 소진 | 클라우드 예산 초과, 발표 당일 서비스 중단 | RA + F6 |

**※ RA: Resource Advisor, GE: Guardrail Engine, PG: Probe Generator, NA: Network Analyzer, VE: Validation Engine, F6: Pipeline Generator**

## 1.3 v1 대비 v2 주요 변경사항

**Grad-Deploy v1에서 v2로의 방향 전환은 단순한 기능 추가가 아닌, 프로젝트의 핵심 가치 재정의입니다. 아래 표는 각 변경 사항과 그 근거를 명시합니다.**

| 구분 | v1 (기존) | v2 (변경) | 변경 근거 |
|---|---|---|---|
| 핵심 타겟 | Docker Compose + K8s 병행 | K8s 전용 집중 | 장애 사례 9건 중 8건이 K8s 운영 오류에서 발생. 범위 집중으로 완성도 확보 (ADR-002) |
| CI/CD 전략 | Jenkins + GitHub Actions | GitHub Actions (CI) + Argo CD (CD) | Jenkins 학습 비용 과다 (R2 리스크). GitOps 방식이 K8s 선언적 철학과 부합 (ADR-001) |
| 핵심 부가가치 | 파일 생성 + GitHub Push | 6대 가드레일 엔진 + GitOps | '생성'에서 '검증+안전 배포'로 가치 확장. 차별화 포인트 강화 |
| 클라우드 연동 | Terraform 프로비저닝 (계획) | 기존 클러스터 연결 방식 (MVP) | 크레딧 소진 리스크 (R3), 구현 복잡도 고려 → Phase 2로 이연 (ADR-003) |
| 백엔드 구조 | 단일 프론트엔드 (React) | Layered Architecture (계층형) | 가드레일 비즈니스 로직 독립성 확보, 팀 분업 최적화 (ADR-010) |

# 2. 요구사항 분석 (Requirement Analysis)

## 2.1 페르소나 정의

**SRS v1.0에 정의된 4개 페르소나를 기반으로, 각 페르소나의 핵심 고충과 설계 시사점을 정리합니다.**

| 페르소나 | 역할 | 핵심 고충 | 설계 시사점 |
|---|---|---|---|
| P1 — 김민수<br>(인프라 초보 학생) | 졸업작품 학생, Docker 경험 있음, K8s 미경험 | YAML 한 줄 오타로 전체 배포 실패. 오류 원인을 스스로 진단 불가. 배포 시도 수십 회 반복 | 1차 UI 가드레일로 즉각 피드백. 오류 메시지에 수정 가이드 인라인 제공. 서비스 유형별 권장값 자동 적용 |
| P2 — 박준영<br>(MSA 팀 리더) | 팀 프로젝트 리더, K8s 기초 지식 보유 | 팀원의 수동 조작(kubectl edit)으로 인한 설정 드리프트. 서비스 간 네트워크 의존성 관리 오류 | GitOps 강제(Config Drift 방지). NetworkPolicy 시각화 + 자동 템플릿 제공. DependencyGraph로 depends_on 시각화 |
| P3 — 이지원<br>(비용 절감 희망자) | 무료 크레딧 사용 중인 학생 개발자 | VM 스펙 답습으로 Resource Request 과도 설정 → 크레딧 조기 소진. 발표 당일 서비스 중단 경험 | RA 엔진: 서비스 유형별 권장 Request 자동 제시. 크레딧 소진 예측 D-day 알림 (F-06) |
| P4 — 심사위원<br>(지도교수 / 심사위원) | 배포 완성도를 실시간으로 평가해야 하는 교수진 | 배포된 서비스의 상태를 실시간으로 확인할 방법 없음 | 데모데이 모드: 공개 URL 자동 발급, 아키텍처/컨테이너 상태/업타임 실시간 대시보드 |

## 2.2 핵심 유저 스토리 및 인수 조건 (AC)

**Value vs Effort 매트릭스를 기준으로 Must / Should / Could로 분류한 유저 스토리 목록입니다. 인수 조건은 측정 가능한 기준으로 작성하여 모호성 안티패턴을 방지합니다.**

| 스토리 ID | 유저 스토리 | 우선순위 | 핵심 인수 조건 |
|---|---|---|---|
| US-01 | As a 학생 개발자, 서비스 템플릿을 선택하면 K8s 매니페스트가 즉시 생성되도록 한다 | Must | · 서비스 선택 후 1초 이내 YAML 미리보기 갱신<br>· Deployment/Service/ConfigMap/Secret/PVC 자동 생성<br>· 서비스 간 envInjection 자동 삽입 |
| US-02 | As a 학생 개발자, 설정 입력 시 가드레일 엔진이 위험을 실시간으로 탐지·차단한다 | Must | · ERROR 수준: 배포 버튼 비활성화 + 항목별 오류 표시<br>· WARNING 수준: 경고 배너 + 권장값 인라인 안내<br>· CPU Request 미설정 시 즉시 WARNING 표시 |

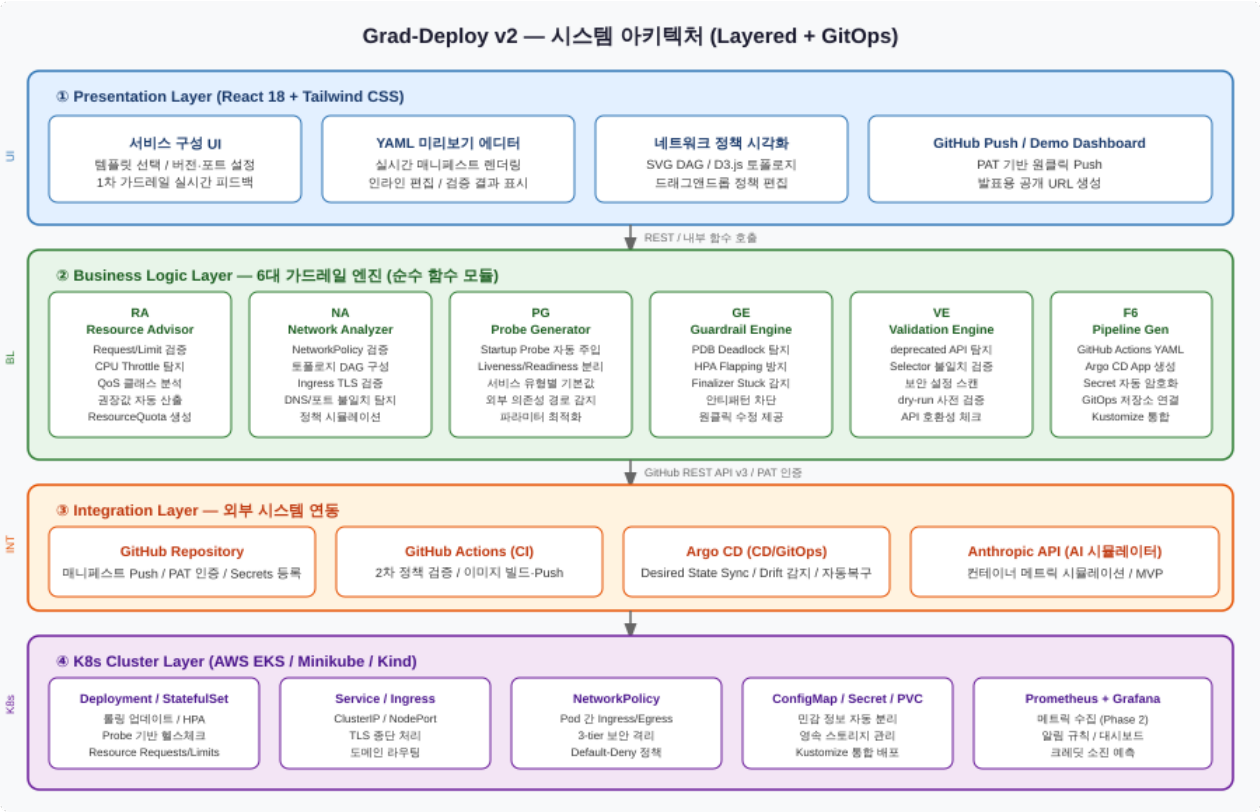
| 스토리 ID | 유저 스토리 | 우선순위 | 핵심 인수 조건 |
|---|---|---|---|
| US-03 | As a 팀 리더, 검증된 매니페스트가 GitHub에 Push되면 Argo CD가 자동 동기화한다 | Must | · GitHub Push 완료 후 30초 이내 Argo CD Sync 시작<br>· OutOfSync 발생 시 대시보드에 즉시 표시<br>· Self-healing: Drift 감지 후 자동 복구 |
| US-04 | As a 학생 개발자, GitHub PAT 하나로 레포지토리 생성 및 파일 Push가 완료된다 | Must | · Fine-grained token 입력 시 Classic token 재입력 유도<br>· 기존 파일 SHA 조회 후 PUT 덮어쓰기<br>· PAT는 sessionStorage/localStorage/Cookie 미저장 강제 |
| US-05 | As a 팀 리더, 서비스 간 네트워크 정책을 시각적으로 설정하고 YAML을 확인한다 | Should | · Frontend→Backend→DB 3-tier 토폴로지 자동 표시<br>· 정책 변경 즉시 NetworkPolicy YAML 미리보기 갱신<br>· default-deny 등 4종 템플릿 원클릭 적용 |
| US-06 | As a 비용 절감 희망자, 크레딧 소진 예측일과 D-day를 비교하여 위험을 사전에 인지한다 | Should | · 잔여 크레딧 ÷ 일 사용량 = 예측 소진일 자동 계산<br>· D-day 대비 '충분/위험' 상태 배지 표시<br>· 자동 슬립 스케줄 생성 기능 |
| US-07 | As a 심사위원, 공개 URL 하나로 팀의 배포 상태를 실시간으로 확인한다 | Must | · <project-slug>-<random5> 형식 공개 URL 자동 발급<br>· 아키텍처 다이어그램/컨테이너 상태/업타임 실시간 표시<br>· LIVE 인디케이터 + 클립보드 복사 버튼 |

# 3. 시스템 설계 (System Design)

## 3.1 아키텍처 다중 관점 (Multi-View)

### 3.1.1 소프트웨어 내부 구조 — 계층형 아키텍처 (Layered Architecture)

**가드레일 비즈니스 로직의 독립성을 확보하고 팀 분업을 최적화하기 위해 Presentation → Business Logic → Integration → K8s Cluster의 4계층 구조를 채택합니다. 각 계층은 단방향 의존(상위 → 하위)으로 연결되며, 계층 간 인터페이스를 통해 결합도를 최소화합니다.**




*[그림 1] Grad-Deploy v2 시스템 아키텍처 (4계층 Layered + GitOps)*

### 3.1.2 계층별 책임 분리 (SoC — Separation of Concerns)

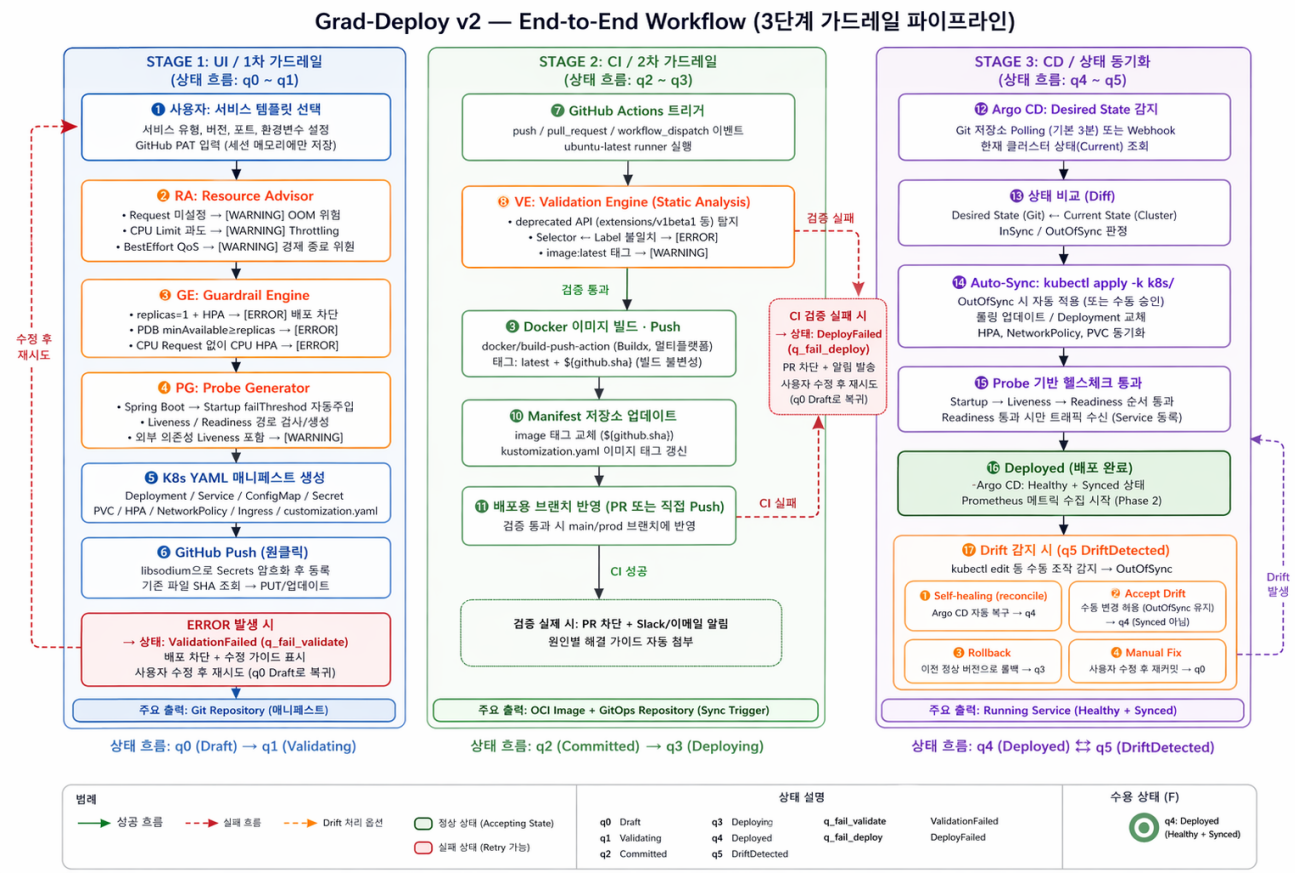
| 계층 | 구성 요소 | 핵심 책임 | 의존 방향 |
|---|---|---|---|
| ① Presentation | React 18 + Tailwind CSS / D3.js | 사용자 입력 수집, 1차 가드레일 실시간 피드백, YAML 미리보기, 네트워크 토폴로지 시각화, GitHub Push UI | ↓ Business Logic 호출 (단방향) |
| ② Business Logic | 6대 가드레일 엔진 (RA/NA/PG/GE/VE/F6) | 정책 검증, 매니페스트 생성, 보안 스캔, 리소스 권장값 계산, 파이프라인 YAML 생성 | ↓ Integration Layer 호출 (단방향) |
| ③ Integration | GitHub REST API / Argo CD / Anthropic API | 외부 시스템 연동, PAT 기반 인증, 이미지 Push, GitOps 동기화 트리거 | ↓ K8s Cluster 조작 (단방향) |
| ④ K8s Cluster | EKS / Minikube / Kind | 실제 컨테이너 오케스트레이션, 상태 유지, 헬스체크, 스케일링 | (최하위 계층) |

### 3.1.3 시스템 배포 패턴 — GitOps 아키텍처

Git 저장소를 '진실의 단일 원천(Single Source of Truth)'으로 삼아, Argo CD가 선언적으로 실제 상태를 관리하는 클라우드 네이티브 패턴을 채택합니다. 이를 통해 Config Drift(Case 5) 문제를 구조적으로 해결합니다. 모든 변경은 Git을 통한 반영을 원칙으로 하며, 수동 변경은 Drift로 탐지되어 자동 복구 대상이 됩니다.

## 3.2 전체 시스템 워크플로우 (End-to-End Workflow — 3단계 가드레일 파이프라인)

Grad-Deploy v2는 사용자의 설정 입력부터 K8s 클러스터에 배포되어 동기화 완료까지, 신뢰할 수 있는 매니페스트만 통과시키는 3단계 가드레일 파이프라인을 구축합니다. 각 단계에서 오류가 발생하면 즉시 차단하고 원인과 수정 방법을 안내합니다.




*[그림 2] Grad-Deploy v2 End-to-End Workflow (3단계 가드레일 파이프라인)*

### STAGE 1 — UI / 1차 가드레일 (Presentation Layer)

사용자는 대시보드에서 서비스 템플릿을 선택하고 설정을 입력합니다. 입력 즉시 RA·GE·PG 엔진이 정책 위반 여부를 검사하는 Validating(q1) 상태로 진입합니다.

실패 시:
ERROR 수준의 정책 위반 발생 시 ValidationFailed(q_fail_validate) 상태로 전이됩니다. 이때 배포 버튼은 즉시 비활성화되며, 사용자가 원인을 파악할 수 있도록 항목별 오류 메시지와 수정 가이드가 인라인으로 표시됩니다. 사용자는 수정을 통해 Draft(q0) 상태로 복귀하여 재검증을 시도해야 합니다.

성공 시:
검증 통과 후 Kubernetes 매니페스트가 생성되고 GitHub에 반영되면 Committed(q2) 상태에 도달합니다.

Stage 1 종료 시점:
Committed(q2) 상태 도달

### STAGE 2 — CI / 2차 가드레일 (GitHub Actions)

Stage 2는 Stage 1의 브라우저 기반 사전 검증과 달리, 실제 배포 기준의 정적 분석 및 실행 검증을 수행하는 Authoritative(결정적) 단계입니다. GitHub Push 이벤트가 발생하면 CI/CD 파이프라인이 트리거되며 Deploying(q3) 상태에 진입합니다.

실패 시:
VE 엔진의 정적 분석 실패, 이미지 빌드 오류, 배포 동기화 실패 등 파이프라인 전반의 실패가 발생하면 DeployFailed(q_fail_deploy) 상태로 전이됩니다. 이때 PR(Pull Request)은 자동으로 차단되며, 사용자에게는 실패 원인과 해결 가이드가 포함된 알림이 즉시 발송됩니다. 사용자는 수정 후 Draft(q0) 상태로 복귀하여 다시 배포를 준비해야 합니다.

성공 시:
모든 검증 및 빌드가 성공하면 Manifest 저장소의 이미지 태그가 갱신되며, 이후 CD 단계에서 동기화가 수행될 준비 상태가 됩니다.

Stage 2 종료 시점:
Deploying(q3) 상태 유지 (동기화 대기)

### STAGE 3 — CD / 상태 동기화 (Argo CD + GitOps)

Argo CD가 변경을 감지하면 동기화(Sync)가 시작되며, 실제 리소스 적용 및 롤아웃이 진행되는 Deploying(q3) 과정이 이어집니다.

배포 완료 조건:
다음 조건을 모두 만족할 경우에만 Deployed(q4) 상태로 간주합니다.
- 모든 Pod가 Ready 상태
- Liveness/Readiness Probe 통과
- Git(Desired State)과 Cluster(Current State)가 일치 (Synced)
- Argo CD 기준: Healthy + Synced

Drift 발생 및 처리:
배포 완료 이후 수동 변경(kubectl edit 등)으로 상태 불일치가 발생하면 DriftDetected(q5) 상태로 전이됩니다. 이후 다음과 같은 대응이 가능합니다:

- reconcile: Argo CD의 Self-healing을 통한 자동 복구 수행
- accept_drift: 운영자가 현재 클러스터 상태를 기준으로 Git 저장소를 수동 업데이트하여 동기화 복구
- rollback: 이전 정상 커밋으로 Git 상태를 롤백하고 CI 트리거를 통해 파이프라인 재가동
- manual_fix: 설정 전면 수정이 필요한 경우 Draft(q0) 상태로 복귀하여 재구성

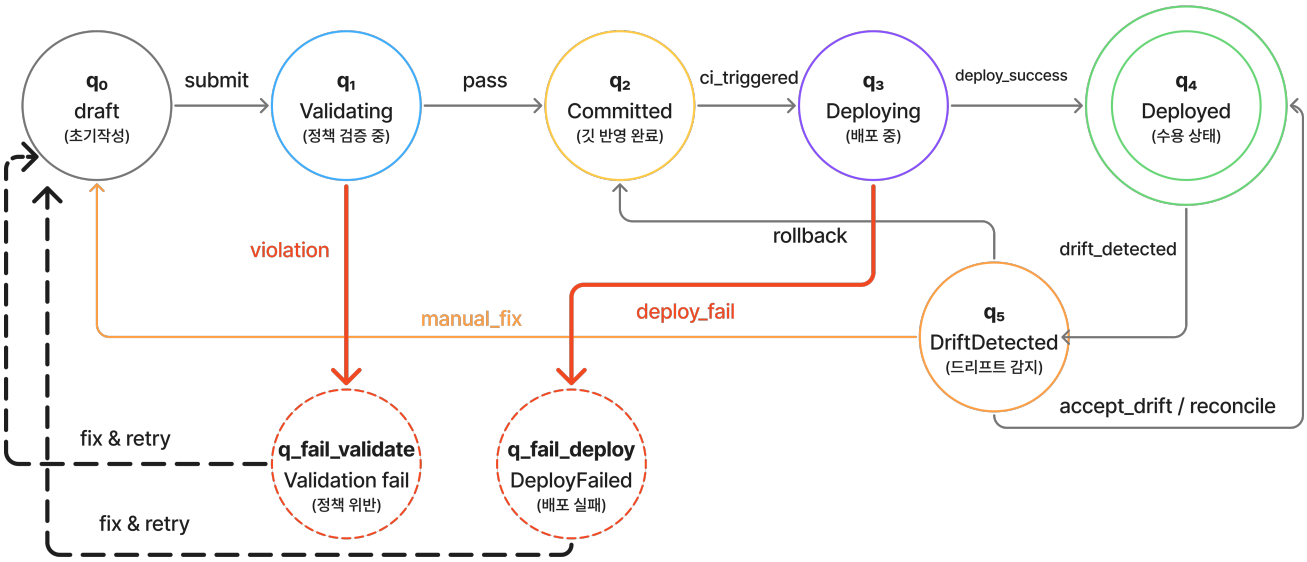
Stage 3 종료 시점:
Deployed(q4) 상태 도달 (최종 수용 상태)

## 3.3 상태 전이도 (State Transition Diagram)

매니페스트의 정책 검증 생명주기를 형식적으로 모델링하기 위해 상태 전이도를 설계하였습니다. 이 모델은 런타임 장애를 모두 막는 만능 도구가 아니라, '배포 전 설정 정책 준수 상태'를 확정적으로 제어하고 Config Drift를 자동 복구하는 데 목적이 있습니다. 본 모델은 Deterministic Finite State Machine(FSM) 기반으로 정의되며, $M = (Q, \Sigma, \delta, q_0, F)$ 형태로 표현됩니다.

*[그림 3]* 매니페스트 생명주기 상태 전이도

**[매니페스트 생명주기 상태 전이 모델 (Partial DFA / Reactive FSM)]**




| 요소 | 정의 |
|---|---|
| Q (상태 집합) | Q = {<br>q0: Draft,<br>q1: Validating,<br>q2: Committed,<br>q3: Deploying,<br>q4: Deployed,<br>q5: DriftDetected,<br>q_fail_validate: ValidationFailed,<br>q_fail_deploy: DeployFailed<br>} |

| 요소 | 정의 |
|---|---|
| Σ (입력 이벤트) | Σ = {<br>submit,<br>pass,<br>violation,<br>ci_triggered,<br>deploy_success,<br>deploy_fail,<br>drift_detected,<br>reconcile,<br>accept_drift,<br>rollback,<br>manual_fix<br>} |
| $q_0$ (초기 상태) | Draft — 사용자가 서비스 구성을 시작한 초기 상태 |
| F (수용 상태) | F = { $q_4$: Deployed }<br>조건:<br>- 모든 Pod가 Ready 상태<br>- Liveness/Readiness Probe 통과<br>- Desired State(Git)와 Current State(Cluster)가 일치 (Synced)<br>- → Argo CD 상태 기준: Healthy + Synced |

## 핵심 전이 함수 (δ) 상세

| 전이 | 이벤트 (Σ) | 조건 | 결과 |
|---|---|---|---|
| δ($q_0$, submit) → $q_1$ | submit | 사용자가 '검증' 또는 '생성' 요청 | 가드레일 엔진(RA/GE/PG) 실행 시작. Validating 상태진입 |
| δ($q_1$, violation) → q_fail_validate | violation | RA/GE/PG에서 ERROR 수준 규칙 위반 탐지 | 배포 차단. UI에 항목별 오류 + 수정 가이드 표시 |
| δ($q_1$, pass) → $q_2$ | Pass | 모든 1차 가드레일 정책 통과 (ERROR 없음) | 매니페스트 생성 및 GitHub Push 완료, Committed 상태 진입 |
| δ(q_fail_validate, fix & retry) → $q_0$ | fix & retry | 사용자가 오류 수정 후 재시도 | Draft 상태로 복귀하여 재검증 대기 |

| 전이 | 이벤트 (Σ) | 조건 | 결과 |
|---|---|---|---|
| $\delta(q_2, ci_triggered) \rightarrow q_3$ | ci_triggered | GitHub Push 감지 후 CI/CD 파이프라인 가동 | VE 정적 분석, 이미지 빌드 및 Argo CD 동기화 시작, Deploying 상태 진입 |
| $\delta(q_3, deploy_fail) \rightarrow q_fail_deploy$ | deploy_fail | CI/CD 파이프라인 전반 실패 (CI/Sync/Probe), 상세 원인은 로그로 구분 | DeployFailed 상태 전이, PR 차단 및 알림 발송 |
| $\delta(q_3, deploy_success) \rightarrow q_4$ | deploy_success | CI 통과 AND Argo CD Sync 성공 AND Probe 통과 | Deployed 상태 (수용 상태) 달성 |
| $\delta(q_fail_deploy, fix\ \&\ retry) \rightarrow q_0$ | fix & retry | 오류 원인 수정 후 재시도 | Draft 상태로 복귀하여 파이프라인 재가동 대기 |
| $\delta(q_4, drift_detected) \rightarrow q_5$ | drift_detected | 클러스터 상태와 Git 상태 불일치 발생 | DriftDetected 상태 전이 |
| $\delta(q_5, reconcile) \rightarrow q_4$ | reconcile | Argo CD Self-healing 수행 및 성공 | Git 상태 기준으로 복구, Deployed 상태 복귀 |
| $\delta(q_5, accept_drift) \rightarrow q_4$ | accept_drift | 운영자가 현재 상태 기준으로 Git 업데이트 | Git-Cluster 동기화 완료, Deployed 상태 복귀 |
| $\delta(q_5, rollback) \rightarrow q_2$ | rollback | 이전 정상 커밋으로 Git 롤백 | Committed 상태 전이 후 CI 트리거로 파이프라인 재시작 |
| $\delta(q_5, manual_fix) \rightarrow q_0$ | manual_fix | 수동 재설정 필요 (복구/롤백 불가) | Draft 상태로 복귀하여 전체 재진행 |

# 4. 기술적 의사결정 기록 (ADR — Architecture Decision Records)

**모든 주요 아키텍처 결정은 '정답은 없다, 오직 트레이드오프만 있다'는 원칙 하에 배경-선택지-결정-이유-대가의 형식으로 기록합니다. ADR은 팀원 간 합의의 근거이자 향후 변경 시 참조 기준입니다.**

| [ADR-001] CI/CD 파이프라인: GitHub Actions + Argo CD 선택 | |
|---|---|
| 상태 | 승인됨 |
| 배경 | v1에서 GitHub Actions 기반 CI/CD가 구현되어 있었고, v2 계획에서 Jenkins를 추가하려 했으나, K8s 클러스터 내 Jenkins 운영 부담과 팀의 Jenkins 학습 비용이 과도하다 판단함 (SRS 리스크 R2). 또한 GitOps 패턴이 K8s의 선언적 철학과 자연스럽게 부합함을 확인함. |
| 선택지 | ① Jenkins + GitHub Actions 혼용 (v2 초기 계획)<br>② GitHub Actions (CI) + Argo CD (CD) — GitOps<br>③ GitHub Actions 단독 (CD 없음) |
| 결정 | **GitHub Actions(CI) + Argo CD(CD) — GitOps 아키텍처 채택** |
| 이유 | · 기존 코드의 Kustomize 기반 K8s 파일 구조(F-11)와 Argo CD가 즉시 통합 가능<br>· GitOps는 Git을 SSOT(Single Source of Truth)로 삼아 Config Drift(Case 5)를 구조적으로 해결<br>· Jenkins 대비 별도 인프라(Helm 배포, 초기 플러그인 설치) 없이 동작하여 운영 부담 Zero<br>· :latest + :${github.sha} 이중 태그로 불변 롤백 기준 확보<br>· workflow_dispatch로 수동 실행 가능, push/pull_request 모두 지원 |
| **대가(Trade-off)** | · Jenkins의 복잡한 파이프라인 커스터마이징(조건부 스테이지, 플러그인 에코시스템) 불가<br>· Argo CD 초기 설정(클러스터 접근 권한, App of Apps 구성)이 단순 Jenkins 대비 초기 복잡성 높음<br>· Argo CD가 동작하는 클러스터가 이미 준비되어 있어야 한다는 선제 조건 존재 |

| [ADR-002] 핵심 범위 집중: Docker Compose 지원 제외 | |
|---|---|
| 상태 | 승인됨 |
| 배경 | v1에서 Docker Compose와 K8s를 동시 지원했으나, 실무 장애 사례 분석 결과(9건 중 8건) 학생 개발자의 실제 장애 포인트가 K8s 설정 오류에 집중됨. Docker Compose 유지 시 6대 가드레일 엔진의 완성도가 분산됨. |
| 선택지 | ① Docker Compose + K8s 동시 지원 유지 (v1 방식)<br>② K8s 전용 집중 + 가드레일 기능 고도화 |
| 결정 | **K8s 전용 집중. Docker Compose는 Phase 2 이후 재검토.** |

| | |
|---|---|
| **이유** | · 단일 도메인 집중으로 가드레일 엔진(RA/NA/PG/GE/VE) 완성도 확보 가능<br>· YAGNI(You Aren't Gonna Need It) 원칙: MVP에서 불필요한 기능 제거<br>· SRS 1.2 시스템 경계 표 기준으로 Docker Compose를 OUT-OF-SCOPE로 명시 이동 가능<br>· K8s 전용 설계로 Kustomize 통합, HPA, NetworkPolicy, StatefulSet 등 고급 기능 완성도 확보 |
| **대가(Trade-off)** | · Docker Compose 환경(로컬 개발 중심 사용자)은 지원 범위에서 제외됨<br>· v1 호환성 일부 포기. 기존 Docker Compose 생성 기능 코드 리팩토링 필요<br>· 로컬 Minikube/Kind 환경에서만 테스트 가능한 일부 K8s 기능 존재 |

| **[ADR-003] 클라우드 프로비저닝: Terraform MVP 제외** | |
|---|---|
| **상태** | **승인됨** |
| **배경** | v2 SRS N-01~N-04에서 Terraform을 통한 K8s 클러스터 자동 프로비저닝을 Phase 1 목표로 계획했으나, 실제 개발 일정과 팀 역량 고려 시 6대 가드레일 엔진과의 병행 개발이 불가함. AWS EKS 클러스터 생성/삭제 반복 테스트는 클라우드 크레딧 소진 리스크(R3) 초래. |
| **선택지** | ① Terraform 프로비저닝 MVP 포함 (원안)<br>② 기존 클러스터 연결 방식 MVP + Terraform 설계만 문서화<br>③ Terraform 완전 제외 (항구적) |
| **결정** | **기존 클러스터(Minikube / 외부 K8s) 연결 방식으로 MVP 구성. Terraform은 Phase 2에서 구현.** |
| **이유** | · SRS SC-04 제약('K8s 모드는 로컬 클러스터 또는 외부 클러스터 연결로 동작')에 근거 확보<br>· 가드레일 엔진의 완성도가 클러스터 자동 생성보다 사용자 가치 측면에서 우선함<br>· 크레딧 소진 리스크(R3) 회피. 개발 환경 안정성 확보<br>· ③ 옵션(완전 제외)이 아닌 ②를 선택하여 Phase 2 연속성 유지. Terraform 모듈 설계는 문서화 |
| **대가(Trade-off)** | · 클러스터 생성 자동화라는 차별화 포인트 일부 지연. v2 SRS 로드맵과의 괴리 발생<br>· 사용자가 직접 클러스터를 준비해야 하는 진입 장벽 잔존<br>· Terraform 없이는 멀티 클라우드 프로비저닝 기능 시연 불가 (발표 시 설명 필요) |

| **[ADR-004] 핵심 엔진 설계: 6대 가드레일 모듈화 (SRP 적용)** | |
|---|---|
| **상태** | **승인됨** |

| | |
|---|---|
| **배경** | 9대 실무 장애 사례에 대응하는 정책 검증 로직을 단일 함수에 구현할 경우, 규칙 추가/수정 시 전체 로직 재검토 필요. 모듈 간 결합도 증가로 유지보수성 저하 우려. |
| **선택지** | ① 단일 검증 함수로 모든 정책 통합 처리<br>② 도메인별 6개 독립 엔진으로 모듈화 (RA/NA/PG/GE/VE/F6)<br>③ 외부 Policy Engine 도입 (OPA/Kyverno) |
| **결정** | **6대 가드레일 엔진으로 모듈화. 각 엔진은 단일 책임 원칙(SRP)을 적용하며 순수 함수로 구현.** |
| **이유** | · SRP(단일 책임): 각 엔진이 하나의 도메인 규칙만 담당 → 규칙 추가 시 해당 엔진만 수정<br>· OCP(개방-폐쇄): 새 서비스 유형 추가 시 SERVICE_TEMPLATES 확장만으로 기존 엔진 수정 없이 자동 반영 (NF-08)<br>· 엔진별 단위 테스트 독립 실행 가능. 병렬 검증으로 응답 속도 향상 가능<br>· Rule Repository(YAML/JSON) 기반으로 규칙을 코드 외부에서 관리 가능 |
| **대가(Trade-off)** | · 6개 엔진 간 입출력 계약(인터페이스) 설계 비용 발생<br>· Result Aggregator 레이어 추가 필요 (severity 기준 정렬, 중복 제거)<br>· ③ OPA/Kyverno 대비: 외부 Policy Engine은 K8s 클러스터 의존성 발생 → MVP 단계 부적합 |

| **[ADR-005] 클라우드 범위: AWS EKS 우선 지원** | |
|---|---|
| **상태** | **승인됨** |
| **배경** | 다중 클라우드(AWS/GCP/Azure/Ncloud)를 동시에 지원할 경우, 각 프로바이더별 StorageClass, LoadBalancer, IAM 차이로 인한 정책 분기 복잡도가 급증함. MVP 단계에서 깊이 있는 정책 모델 수립이 우선. |
| **선택지** | ① AWS EKS + GCP GKE 동시 지원<br>② AWS EKS 우선 지원 + 로컬(Minikube/Kind) 동시 지원<br>③ 클라우드 무관 추상화 레이어 설계 |
| **결정** | **AWS EKS 우선 지원 + 로컬 클러스터(Minikube/Kind) 동시 지원. GCP/Azure는 Phase 2.** |
| **이유** | · 가장 표준적인 관리형 K8s(EKS)에서 깊이 있는 정책 모델 수립 가능<br>· Minikube/Kind 병행 지원으로 크레딧 없이도 개발·테스트 환경 구성 가능<br>· EKS 기반 StorageClass(gp2/gp3), IAM 정책을 기준으로 VE 검증 규칙 명확화 |
| **대가(Trade-off)** | · AWS 종속 설정(gp2 StorageClass 등)이 GCP/Azure 환경과 호환 안 될 수 있음<br>· 멀티클라우드 시연 불가 → 발표 시 범위 제한 명시 필요 |

| [ADR-006] 인증 및 데이터 전략: GitHub PAT 기반 무상태(Stateless) 설계 | |
|---|---|
| 상태 | 승인됨 |
| 배경 | 서버 측 DB를 운영하면 PAT 등 민감 정보의 저장·암호화·접근 제어 부담이 발생. MVP 단계에서 인증·권한 관리 시스템은 OUT-OF-SCOPE(SRS SC-03)로 명시됨. |
| 선택지 | ① 서버 DB에 사용자 정보·PAT 저장 (세션 관리)<br>② GitHub PAT 기반 무상태 설계 (브라우저 세션 메모리에만 보관)<br>③ GitHub OAuth 앱 연동 (인가 코드 흐름) |
| 결정 | **GitHub Classic PAT를 브라우저 세션 메모리에만 보관. 서버 측 DB 미운영. 페이지 새로고침 시 PAT 즉시 소멸.** |
| 이유 | · 서버 측 DB 운영 리스크 완전 제거 (NF-03 보안 요구사항 충족)<br>· 사용자 데이터 주권 보장 (Privacy-first 설계). sessionStorage/localStorage/Cookie 미저장 검증 로직 포함<br>· autocomplete='off' 적용으로 브라우저 자동완성 통한 값 노출 방지 (SC-02)<br>· Secrets 등록 시 libsodium sealed box(crypto_box_seal)로 브라우저 내 암호화 후 전송 (NF-11) |
| **대가(Trade-off)** | · 브라우저 새로고침 시 PAT 재입력 필요 → UX 불편<br>· Fine-grained token 미지원 (Classic PAT만 허용) → repo + workflow scope 필수 안내 필요<br>· 단일 사용자 세션 기반 MVP → 멀티 사용자 동시 편집은 Phase 2 대응 (SC-03) |

| [ADR-007] 프론트엔드 기술: React 18 + D3.js / SVG 시각화 | |
|---|---|
| 상태 | 승인됨 |
| 배경 | 네트워크 토폴로지 맵, 의존성 그래프(DAG), 상태 전이 시각화 등 복잡한 인터랙티브 그래픽이 핵심 UI 요소임. 기존 v1 코드베이스가 React 기반으로 구현되어 있어 연속성 확보 필요. |
| 선택지 | ① React 18 + D3.js (데이터 주도 시각화)<br>② Vue 3 + ECharts<br>③ Angular + Chart.js |
| 결정 | **React 18 + Tailwind CSS (UI 구성) + D3.js / 인라인 SVG (토폴로지 시각화). useReducer 패턴으로 복잡한 서비스 상태 관리.** |

| | |
|---|---|
| **이유** | · 기존 v1 DependencyGraph 컴포넌트(SVG 기반)를 확장하여 NetworkPolicy 시각화에 재사용 가능<br>· D3.js의 force-directed graph로 서비스 디스커버리 토폴로지 맵 구현 가능<br>· useReducer + ACTION 패턴으로 ADD_SERVICE/REMOVE_SERVICE/ADD_CONNECTION/UPDATE_VERSION_PORT 등 복잡한 상태 관리 검증됨 (기존 코드 검증 완료)<br>· Tailwind CSS로 반응형 UI 구성. 모바일 최소 열람 지원 (NF-07) |
| **대가(Trade-off)** | · D3.js 학습 곡선이 높음. 복잡한 force 레이아웃 커스터마이징에 시간 소요 가능성<br>· 대형 서비스 그래프(20개 이상 노드) 렌더링 성능 최적화 필요 |

| **[ADR-008] 백엔드 기술: Spring Boot 3 (Phase 2 계획)** | |
|---|---|
| **상태** | **승인됨** |
| **배경** | MVP 단계에서는 순수 프론트엔드(React)로 동작하지만, Phase 2에서 클라우드 빌링 API 프록시, Docker Socket 실제 모니터링, 공개 URL 라우팅 등 백엔드 서버가 필요함. 기술 스택을 사전 결정하여 설계 연속성 확보. |
| **선택지** | ① Spring Boot 3 (REST API + K8s Java Client)<br>② Node.js (Express / Fastify)<br>③ Python FastAPI |
| **결정** | **Phase 2 백엔드: Spring Boot 3. K8s Java Client 라이브러리를 통한 클러스터 연동. MVP 단계에서는 브라우저에서 GitHub REST API 직접 호출.** |
| **이유** | · K8s 공식 Java Client(io.kubernetes:client-java)의 타입 안전성 및 성숙도 높음<br>· Spring Boot의 계층형 아키텍처 강제 → 비즈니스 로직 독립성 확보 (ADR-010과 일관성)<br>· SRS 4.1 기술 스택 표에 이미 Spring Boot / Node.js(Credit Server) 병행 명시됨<br>· Prometheus Actuator 연동으로 메트릭 노출 용이 (Phase 2 모니터링 스택 연동) |
| **대가(Trade-off)** | · Spring Boot 기동 시간이 느림 → Startup Probe failureThreshold 높게 설정 필요 (PG 엔진이 자동 처리)<br>· Node.js 대비 메모리 사용량 높음 → 개발 환경에서 리소스 제약 고려 필요 |

| **[ADR-009] 보안 설계: Anthropic API Key 마스킹 및 세션 격리** | |
|---|---|
| **상태** | **승인됨** |
| **배경** | AI 시뮬레이션 모드(F-04)에서 사용하는 Anthropic API Key가 화면·네트워크·콘솔 로그에 노출될 경우 심각한 보안 사고 발생 가능. 두 종류의 민감 정보(GitHub PAT, Anthropic API Key)를 각각 다르게 취급해야 함. |

| | |
|---|---|
| **선택지** | ① 서버에 API Key 저장 후 서버 프록시 경유 호출<br>② 브라우저 세션 메모리 보관 + UI 마스킹 + 콘솔 로그 제거<br>③ LocalStorage 저장 (편의성 우선) |
| **결정** | **Anthropic API Key: password 타입 필드(UI 마스킹), 세션 메모리 보관, 네트워크 페이로드·콘솔 로그 노출 금지. autocomplete='off' 적용. (NF-04 준수)** |
| **이유** | · NF-04: 'Anthropic API Key는 UI 비밀번호 필드로만 입력. 콘솔 로그·네트워크 페이로드에 노출 금지' 요구사항 직접 충족<br>· Phase 2에서 백엔드 프록시 도입 시 서버 사이드 보관으로 자연스럽게 전환 가능<br>· 사용자 데이터 주권 보장: 키 값이 서버에 전혀 저장되지 않음 |
| **대가(Trade-off)** | · 브라우저 새로고침 시 API Key 재입력 필요 → 모니터링 세션 지속 불가<br>· 클라이언트 사이드 직접 호출이므로 CORS 정책에 의존 → Anthropic API의 CORS 허용 여부 사전 검증 필요 |

| **[ADR-010] 소프트웨어 구조 패턴: 계층형 아키텍처(Layered Architecture) 선택** | |
|---|---|
| **상태** | **승인됨** |
| **배경** | Grad-Deploy의 핵심은 6대 가드레일 비즈니스 로직이다. 이 로직의 독립성을 확보하고, 팀원이 병렬로 개발할 수 있는 구조가 필요하다. 복잡한 분산 처리보다 관심사의 분리(SoC)와 팀 분업이 우선한다. |
| **선택지** | ① 계층형(Layered) — Presentation → Business Logic → Integration → K8s<br>② MSA (마이크로서비스) — 엔진별 독립 서비스화<br>③ 모듈러 모놀리스 — 패키지 기반 경계 |
| **결정** | **계층형 아키텍처 채택. 4개 계층: Presentation → Business Logic (6대 엔진) → Integration → K8s Cluster. 상위→하위 단방향 의존.** |
| **이유** | · 가드레일 비즈니스 로직(6대 엔진)이 Integration, K8s에 의존하지 않아 단위 테스트 독립 실행 가능 (DIP 원칙)<br>· 팀원 분업: Presentation 담당자 / Business Logic 담당자 / Integration 담당자로 병렬 개발 가능<br>· 요청-응답의 직관적 흐름으로 디버깅 용이. 싱크홀 안티패턴은 단순 조회에 한해 캐시로 완화 예정<br>· OCP 적용: SERVICE_TEMPLATES에 신규 서비스 추가 시 기존 엔진 코드 수정 없이 자동 반영 (NF-08) |

| | |
|---|---|
| **대가(Trade-off)** | · 모듈 간 물리적 분리가 아니므로, 패키지 경계만으로 결합도를 통제해야 함 → 코드 리뷰 규율 필요<br>· Phase 2에서 트래픽 증가 시 특정 계층(예: NA 엔진)만 스케일아웃하기 어려움 → MSA 전환 가능성 열어둠<br>· 6대 엔진을 단일 Business Logic 계층에 포함하므로, 향후 엔진이 20개 이상으로 증가하면 계층 내 복잡도 증가 |

# 5. 가드레일 상세 로직 (Guardrail Logic Specification)

**본 섹션은 6대 가드레일 엔진의 탐지 규칙, 판단 로직, 출력 형식을 상세 명세합니다. 각 규칙은 실무 장애 사례 (Section 1.2)와 1:1로 대응합니다.**

## 5.1 RA — Resource Advisor (리소스 어드바이저)

**대응 장애: Case 1(Resource Throttling), Case 2(PDB Deadlock의 전제 조건), Case 9(Cost Explosion)**

| 규칙 ID | 탐지 조건 | 심각도 | 출력 메시지 및 대응 액션 |
|---|---|---|---|
| RA-01 | pod.spec.containers[].resources.requests == null | WARNING | [OOM 위험] CPU/Memory Request가 미설정되어 BestEffort QoS 클래스로 동작합니다. OOM 발생 시 가장 먼저 강제 종료됩니다. → 권장값 자동 제시 |
| RA-02 | resources.requests.cpu > (권장값 × 3) | WARNING | [리소스 과다 예약] 요청된 CPU가 권장값의 300%를 초과합니다. 클라우드 크레딧 조기 소진 위험이 있습니다. → 권장값으로 조정 유도 |
| RA-03 | resources.limits.cpu가 존재하고 limits.cpu < requests.cpu × 2 | WARNING | [Throttling 위험] CPU Limit이 Request의 2배 미만입니다. 트래픽 burst 시 CFS Throttling으로 성능이 급격히 저하될 수 있습니다. |
| RA-04 | Namespace에 ResourceQuota 미설정 | WARNING | [Noisy Neighbor 위험] Namespace 단위 ResourceQuota가 없습니다. 단일 Pod가 클러스터 자원을 독점할 수 있습니다. → ResourceQuota 자동 생성 버튼 제공 |
| RA-05 | 단일 Pod의 memory.limits > Namespace 전체 memory 합계 × 0.5 | WARNING | [Noisy Neighbor 위험] 단일 Pod의 메모리 Limit이 Namespace 전체의 50%를 초과합니다. |

**서비스 유형별 권장 Request 기본값 (RA 엔진 자동 적용)**

| 서비스 유형 | CPU Request | Memory Request | CPU Limit | Memory Limit | 비고 |
|---|---|---|---|---|---|
| Spring Boot | 250m | 512Mi | 미설정 권장 | 1Gi | JVM 힙 메모리 고려. Startup burst 처리를 위해 CPU Limit 미설정 권장 |
| Node.js Backend | 100m | 256Mi | 미설정 권장 | 512Mi | 단일 스레드. CPU Limit 설정 시 I/O 집약 작업에서 Throttling 위험 |

| 서비스 유형 | CPU Request | Memory Request | CPU Limit | Memory Limit | 비고 |
|---|---|---|---|---|---|
| Python Flask/ FastAPI | 100m | 256Mi | 미설정 권장 | 512Mi | GIL 제약으로 CPU 단일 코어 사용. Limit 미설정 권장 |
| React (Nginx) | 50m | 128Mi | 200m | 256Mi | 정적 파일 서빙 특성상 CPU 사용 낮음. Limit 설정 가능 |
| MySQL | 250m | 512Mi | 500m | 1Gi | StatefulSet 배포. 데이터 볼륨 I/O 고려 |
| Redis | 100m | 128Mi | 200m | 256Mi | 메모리 기반 DB. Memory Limit 초과 시 즉시 강제 종료 |
| MongoDB | 250m | 512Mi | 500m | 2Gi | 인덱스 메모리 요구량 높음. Limit 여유롭게 설정 |

## 5.2 GE — Guardrail Engine (가드레일 엔진)

**대응 장애: Case 2(PDB Deadlock), Case 6(Finalizer Stuck), HPA Flapping(Case 1의 변형)**

| 규칙 ID | 탐지 조건 | 심각도 | 출력 메시지 및 대응 액션 |
|---|---|---|---|
| GE-01 | pdb.spec.minAvailable >= deployment.spec.replicas | ERROR | [PDB Deadlock] minAvailable 값이 replicas 수 이상입니다. 노드 드레인 시 Pod를 종료할 수 없어 클러스터 업데이트가 영구 차단됩니다. → 권장값: minAvailable = max(1, replicas - 1) 자동 제안 |
| GE-02 | hpa 설정 + resources.requests.cpu == null | ERROR | [HPA 설정 오류] CPU 기반 HPA가 설정되었으나 CPU Request가 미설정되어 있습니다. K8s는 Request를 기준으로 CPU 사용률을 계산하므로 HPA가 정상 동작하지 않습니다. |
| GE-03 | hpa.spec.behavior.scaleDown.stabilizationWindowSeconds == null | WARNING | [HPA Flapping 위험] Scale Down Stabilization Window가 미설정되어 있습니다. 단기 CPU 스파이크에 반복적으로 반응하여 Pod가 계속 생성·삭제될 수 있습니다. → scaleUp: 60초, scaleDown: 300초 자동 주입 권장 |
| GE-04 | deployment.spec.replicas == 1 AND hpa 설정 | WARNING | [단일 Replica + HPA] HPA 최소 replica가 1이면 HPA가 의미 없습니다. 서비스 가용성을 위해 minReplicas: 2 이상 권장 |
| GE-05 | 리소스 삭제 요청 후 Terminating 상태 5분 이상 (실시간 모드) | WARNING | [Finalizer Stuck] 리소스가 5분 이상 Terminating 상태입니다. Finalizer 해제 실패로 고착된 것으로 판단됩니다. → kubectl patch 명령으로 강제 삭제 옵션 제공 |

## 5.3 PG — Probe Generator (프로브 생성기)

**대응 장애: Case 3(Probe Loop). Liveness/Readiness/Startup Probe의 역할을 강제 분리하여 무한 재시작 루프를 예방합니다.**

| 규칙 ID | 탐지 조건 | 심각도 | 출력 메시지 및 대응 액션 |
|---|---|---|---|
| PG-01 | livenessProbe 미설정 | WARNING | [Liveness Probe 미설정] 컨테이너 비정상 종료 시 자동 재시작되지 않습니다. → 서비스 유형별 기본값 자동 주입 제안 |
| PG-02 | readinessProbe 미설정 | WARNING | [Readiness Probe 미설정] 트래픽 수신 준비가 완료되기 전에 Service에 등록되어 오류 응답을 반환할 수 있습니다. |
| PG-03 | startupProbe 미설정 + Spring Boot/JVM 계열 서비스 | WARNING | [Startup Probe 미설정] JVM 초기 기동(30~60초)이 완료되기 전에 Liveness Probe가 실패 판정하여 무한 재시작 루프 위험이 있습니다. → failureThreshold:30, periodSeconds:10 자동 주입 (최대 5분 대기) |
| PG-04 | livenessProbe.path가 DB 연결 포함 의심 패턴 (예: /health/db, /actuator/health/db) | WARNING | [Cascading Restart 위험] Liveness Probe가 외부 의존성(DB)을 포함합니다. DB 장애 시 불필요한 Pod 재시작이 발생합니다. → Liveness: /healthz (내부만), Readiness: /ready (외부 포함) 분리 권장 |
| PG-05 | probe.spec.periodSeconds < 5 | WARNING | [Probe 주기 너무 짧음] periodSeconds가 5초 미만입니다. 과도한 헬스체크 트래픽이 발생할 수 있습니다. |

**서비스 유형별 Probe 자동 주입 기본값**

| 서비스 | Startup Probe | Liveness Probe | Readiness Probe |
|---|---|---|---|
| Spring Boot | failureThreshold:30, periodSeconds:10<br>(최대 5분 대기) | httpGet: /actuator/health<br>periodSeconds: 30, failureThreshold: 3 | httpGet: /actuator/health/readiness<br>periodSeconds: 10, failureThreshold: 3 |
| Node.js Backend | failureThreshold:10, periodSeconds:5<br>(최대 50초 대기) | httpGet: /healthz<br>periodSeconds: 20, failureThreshold: 3 | httpGet: /ready<br>periodSeconds: 10, failureThreshold: 3 |
| Python Flask/FastAPI | failureThreshold:15, periodSeconds:5<br>(최대 75초 대기) | httpGet: /healthz<br>periodSeconds: 20, failureThreshold: 3 | httpGet: /ready<br>periodSeconds: 10, failureThreshold: 3 |
| React (Nginx) | failureThreshold:5, periodSeconds:3<br>(최대 15초 대기) | httpGet: /<br>periodSeconds: 30, failureThreshold: 3 | httpGet: /<br>periodSeconds: 10, failureThreshold: 3 |

## 5.4 NA — Network Analyzer (네트워크 분석기)

**대응 장애: Case 4(Network Blindness). NetworkPolicy 부재로 인한 자원 노출 및 불필요한 Pod 간 통신을 사전 탐지합니다.**

| 규칙 ID | 탐지 조건 | 심각도 | 출력 메시지 및 대응 액션 |
|---|---|---|---|
| NA-01 | Namespace에 NetworkPolicy 미설정 | WARNING | [전체 통신 허용] NetworkPolicy가 없어 모든 Pod 간 통신이 허용됩니다. DB 등 내부 자원이 외부에 노출될 수 있습니다. → 4종 템플릿 (default-deny / frontend-to-backend / backend-to-db / allow-all-dev) 원클릭 적용 제공 |
| NA-02 | Ingress TLS 설정 미존재 | WARNING | [TLS 미설정] Ingress에 TLS 설정이 없어 HTTP로 트래픽이 전송됩니다. 민감 데이터 노출 위험이 있습니다. |
| NA-03 | Ingress backend service가 실제 Service와 매칭 안 됨 | ERROR | [Ingress 라우팅 오류] Ingress backend의 serviceName이 실제 Service 이름과 일치하지 않습니다. 트래픽이 404를 반환합니다. |
| NA-04 | 순환 의존성(Circular Dependency) 탐지 (A→B→A) | ERROR | [순환 의존성] 서비스 간 순환 의존성이 탐지되었습니다. depends_on 설정으로 인해 데드락 발생 가능합니다. |

## 5.5 VE — Validation Engine (검증 엔진)

**대응 장애: Case 7(PVC Unbound), Case 8(Image Auth Fail). 배포 전 YAML 정적 분석으로 런타임 오류를 사전 차단합니다.**

| 규칙 ID | 탐지 조건 | 심각도 | 출력 메시지 및 대응 액션 |
|---|---|---|---|
| VE-01 | deprecated/removed K8s API 사용 (예: extensions/v1beta1 Ingress) | ERROR | [API 버전 오류] 현재 K8s 클러스터 버전에서 제거된 API입니다. → networking.k8s.io/v1 로의 마이그레이션 자동 안내 |
| VE-02 | Deployment selector.matchLabels ≠ Pod template.metadata.labels | ERROR | [Selector 불일치] Deployment의 selector와 Pod의 label이 일치하지 않습니다. Pod를 찾지 못해 배포가 실패합니다. |
| VE-03 | pvc.spec.storageClassName == null (클라우드 환경) | WARNING | [StorageClass 미지정] 클라우드 환경에서 StorageClass를 지정하지 않으면 PVC가 Pending 상태에서 기동되지 않습니다. → gp2(AWS) 등 환경별 권장값 제시 |
| VE-04 | imagePullSecrets 미설정 + Private Registry 이미지 사용 의심 | WARNING | [인증 정보 누락] Private Registry의 이미지를 사용하지만 imagePullSecrets가 설정되지 않았습니다. ErrImagePull 오류가 발생합니다. |
| VE-05 | container.image 태그가 ':latest' 또는 태그 없음 | WARNING | [latest 태그 사용] 'latest' 태그는 이미지 변경 추적이 불가하여 예기치 않은 버전이 배포될 수 있습니다. → :${github.sha} 불변 태그 사용 권장 |
| VE-06 | container.securityContext.privileged == true | WARNING | [보안 취약] 컨테이너가 Privileged 모드로 실행됩니다. 호스트 시스템 자원에 전체 접근 권한이 부여됩니다. |

# 6. 기능 목록 및 우선순위 (MoSCoW)

**Value vs Effort 매트릭스를 기준으로 Must/Should/Could로 분류합니다. P0/P1/P2는 구현 단계를 의미합니다.**

| 기능 ID | 기능명 | MoSCoW | 단계 | 상태 | 비고 |
|---|---|---|---|---|---|
| F-01 | K8s 매니페스트 자동 생성 (Deployment/Service/ConfigMap/Secret/PVC/HPA/Ingress) | Must | P0 | 기존 확장 | Docker Compose 제외, K8s 전용 |
| F-02 | GitHub Actions 워크플로우 자동 생성 | Must | P0 | 기존 확장 | Argo CD CD 연동 추가 |
| F-03 | GitHub 원클릭 Push (PAT 기반, Secrets 암호화 등록) | Must | P0 | 기존 유지 | libsodium 암호화 포함 |
| F-04 (RA) | Resource Advisor — Request/Limit 검증 및 권장값 자동 제시 | Must | P0 | 신규 | Case 1, 9 대응 |
| F-05 (GE) | Guardrail Engine — PDB Deadlock, HPA Flapping, Finalizer Stuck 탐지 | Must | P0 | 신규 | Case 2, 6 대응 |
| F-06 (VE) | Validation Engine — deprecated API, Selector 불일치, 보안 스캔 | Must | P0 | 신규 | Case 7, 8 대응 |
| F-07 (PG) | Probe Generator — 서비스 유형별 Probe 자동 주입 | Should | P1 | 신규 | Case 3 대응 |
| F-08 (NA) | Network Analyzer — NetworkPolicy 시각화, 4종 템플릿, Ingress 검증 | Should | P1 | 신규 | Case 4 대응 |
| F-09 | DependencyGraph 확장 — K8s NetworkPolicy YAML 연동 | Should | P1 | 기존 확장 | 기존 SVG 그래프 재활용 |
| F-10 | 크레딧 번아웃 방지 (수동 입력 기반 예측·시각화) | Should | P1 | 기존 유지 | API 연동은 Phase 2 |
| F-11 | 데모데이 모드 (공개 URL 자동 발급 + 실시간 대시보드) | Must | P1 | 기존 확장 | 발표 당일 심사위원 접속 지원 |
| F-12 | Terraform 클러스터 프로비저닝 (AWS EKS/GCP GKE) | Could | P2 | Phase 2 | ADR-003: MVP 제외 |
| F-13 | 실제 Docker Socket 연동 모니터링 | Could | P2 | Phase 2 | 현재 AI 시뮬레이터로 대체 |
| F-14 | 클라우드 빌링 API 직접 연동 (AWS Cost Explorer 등) | Could | P2 | Phase 2 | CORS 제약 → 백엔드 프록시 필요 |

# 7. 향후 일정 및 로드맵

## 7.1 개발 로드맵 (페이즈별)

| 단계 | 기간 | 주요 목표 | 핵심 산출물 | 마일스톤 |
|---|---|---|---|---|
| Phase 1<br>(MVP) | 현재~8주 | P0 기능 완성<br>6대 엔진 핵심 로직 구현 | K8s 매니페스트 생성기<br>RA/GE/VE 엔진<br>GitHub Actions + Argo CD 파이프라인 | YAML 생성 → GitHub Push → Argo CD Sync 성공 시연 |
| Phase 1.5<br>(P1 기능) | 9~14주 | P1 기능 추가<br>네트워크 시각화 완성 | PG/NA 엔진<br>DependencyGraph 확장<br>크레딧 관리 UI<br>데모데이 모드 | NetworkPolicy GUI 설정 → YAML 생성 → 배포 시연 |
| Phase 2<br>(고도화) | 15주 이후 | 인프라 자동화<br>백엔드 서버 구축 | Terraform 모듈<br>Spring Boot API 서버<br>실제 모니터링 연동<br>공개 URL 라우팅 | 클러스터 자동 생성 → 배포 → 모니터링 전체 플로우 시연 |

## 7.2 리스크 및 대응 방안

| 리스크 ID | 리스크 | 영향도 | 발생 조건 | 대응 방안 |
|---|---|---|---|---|
| R1 | Terraform 프로바이더 호환성 | 상 | AWS/GCP API 변경 시 Terraform 모듈 브레이킹 | ADR-003: Phase 2로 이연. AWS EKS 우선. lock 파일 버전 고정 |
| R2 | Jenkins 학습 비용 | 중 | 팀원 Jenkins 경험 부족으로 개발 지연 | ADR-001: Argo CD로 대체. 학습 비용 Zero |
| R3 | 클라우드 크레딧 소진 | 상 | Terraform/EKS 실행으로 크레딧 빠른 소모 | ADR-003: MVP에서 Minikube 우선. RA 엔진 자원 최적화 |
| R4 | 가드레일 규칙 과다 탐지 (False Positive) | 중 | 정상 설정에도 WARNING 남발 → 사용자 피로 | WARNING/ERROR 임계값 튜닝. 규칙별 suppression 옵션 제공 |
| R5 | 일정 지연 | 중 | 예상치 못한 기술적 난제 | P0 기능 우선 완성. P1은 여유 시 개발. 4주 버퍼 확보 |

# 8. 추가 고려사항 (Cross-Cutting Concerns)

## 8.1 보안 설계 요약

| 대상 | 보안 요건 | 구현 방법 | 근거 |
|---|---|---|---|
| GitHub PAT | 브라우저 세션 메모리에만 보관 | sessionStorage/localStorage/Cookie 미저장 강제. 새로고침 시 즉시 소멸 검증 | SC-02, NF-03 |
| Anthropic API Key | UI 마스킹 + 콘솔/네트워크 로그 미노출 | password type 필드. autocomplete='off'. 네트워크 페이로드 확인 | NF-04 |
| GitHub Secrets | 평문 전송 금지 | libsodium crypto_box_seal으로 브라우저 내 암호화 후 PUT | NF-11 |
| K8s Secret YAML | Git 저장소 노출 방지 | .gitignore에 *-secret.yaml 패턴 자동 포함 | NF-13 |
| 민감 환경변수 | ConfigMap/Secret 자동 분리 | password/secret/token 패턴 키 → Secret(stringData), 나머지 → ConfigMap | FR-01-14 |

## 8.2 비기능 요구사항 (NF) 검증 계획

| NF ID | 요구사항 | 검증 방법 | 기준 |
|---|---|---|---|
| NF-01 | 서비스 선택 → YAML 생성 10분 이내 | 시나리오 기반 사용성 테스트 (K8s 미경험 사용자 5명) | 평균 완료 시간 < 10분 |
| NF-05 | YAML 생성 1초 이내 (서비스 10개) | 브라우저 성능 프로파일링 (React DevTools) | 클라이언트 연산 < 1,000ms |
| NF-06 | GitHub Push 파일당 평균 3초 이내 | 브라우저 Network 탭 PUT /repos/.../contents/... 응답 시간 기록 | 전체 파일셋(10개) < 30초 |
| NF-07 | Chrome/Firefox/Safari/Edge 최신 버전 지원 | 크로스브라우저 테스트 (BrowserStack 또는 수동) | 4개 브라우저 주요 기능 정상 동작 |
| NF-08 | 신규 서비스 추가 시 기존 코드 수정 없음 | SERVICE_TEMPLATES에 신규 항목 추가 후 UI 자동 반영 확인 | 코드 변경 없이 UI에 반영 |

## 8.3 추적성 체크리스트 (Traceability Matrix — 핵심 항목)

| 유저 스토리 | 유스케이스 (장애 케이스) | 가드레일 엔진 | 핵심 함수/규칙 ID | 출력 결과 |
|---|---|---|---|---|
| US-01 (YAML 생성) | K8s 매니페스트 자동 구성 (FR-01-1~16) | VE | VE-01, VE-02, VE-03 | Deployment/Service/ConfigMap/Secret/PVC YAML |

| 유저 스토리 | 유스케이스 (장애 케이스) | 가드레일 엔진 | 핵심 함수/규칙 ID | 출력 결과 |
|---|---|---|---|---|
| US-02 (가드레일) | Case 1 (Resource Throttling) | RA | RA-01, RA-02, RA-03 | WARNING: CPU Request 권장값 + 조정 유도 |
| US-02 (가드레일) | Case 2 (PDB Deadlock) | GE | GE-01 | ERROR: 배포 차단 + minAvailable 권장값 |
| US-02 (가드레일) | Case 3 (Probe Loop) | PG | PG-03, PG-04 | WARNING: Startup Probe 자동 주입 + 역할 분리 권장 |
| US-02 (가드레일) | Case 4 (Network Blindness) | NA | NA-01, NA-02 | WARNING: NetworkPolicy 템플릿 원클릭 적용 |
| US-03 (GitOps) | Config Drift 방지 (Case 5) | GitOps/ Argo CD | $\delta(q_4$, drift_detected) → $q_5$ | DriftDetected → Self-healing → Deployed |
| US-04 (GitHub Push) | 보안 (SC-02, NF-11) | 보안 모듈 | libsodium crypto_box_seal | 암호화된 Secrets → GitHub Actions Secrets API |
| US-07 (데모데이) | 데모데이 모드 (F-05) | F6 (Pipeline Gen) | FR-05-1~6 | 공개 URL 자동 발급 + 실시간 대시보드 |

# 결론 및 기대 효과

**Grad-Deploy v2는 9대 실무 장애 사례에서 도출한 데이터 기반 문제 정의를 바탕으로, 6대 가드레일 엔진 + GitOps 아키텍처를 통해 학생 개발자가 Kubernetes 환경에서 '안전하게 실패하고 빠르게 성공하도록' 돕는 운영 안전망 플랫폼을 목표로 합니다.**

**핵심 차별화 포인트는 세 가지입니다. 첫째, 기존 K8s 관리 도구(Rancher, Lens, Portainer)가 이미 K8s를 아는 사용자를 대상으로 하는 반면, Grad-Deploy는 K8s를 모르는 학생이 올바른 설정을 갖춘 매니페스트를 생성하도록 Guardrail로 가이드합니다. 둘째, GitOps 아키텍처를 통해 Config Drift 문제를 구조적으로 해결하고 Self-healing을 자동화합니다. 셋째, 9대 실무 장애 사례와 1:1로 대응하는 정책 규칙을 Data-Driven으로 설계하여 실무 신뢰성을 확보합니다.**

**10개의 ADR을 통해 모든 주요 기술 결정의 배경-선택지-결정-이유-대가를 명시하였으며, 이는 팀원 간 합의의 근거이자 향후 변경 시 참조 기준으로 활용됩니다. 추적성 체크리스트를 통해 유저 스토리→가드레일 규칙→출력 결과가 1:1로 연결됨을 검증하였습니다.**